Sprint 12 Full : Performance Foundations, Browser Rendering, INP, and Sustainability

0. Where You Are Before Sprint 12

- **Concept Explanation (Sinhala):** Sprint 12 එකට එන්න කලින්, ඔයා React වල මූලික සංකල්ප ටික හොඳින් තේරුම් අරන් ඉන්න ඕනේ. ඒ කියන්නේ React පාවිච්චි කරලා UI හදන හැටි, Vite වලින් project එකක් setup කරන හැටි, JSX ලියන හැටි, සහ parent component එකක ඉඳන් child component එකකට Props හරහා data pass කරන හැටි වගේ දේවල්. ඒ වගේම useState, useReducer වැනි Hooks හරහා state change කරන හැටිත්, useEffect වලින් side effects handle කරන හැටිත් ඔයා ඉගෙනගෙන ඇති. මීට කලින් Sprint 8 වලදී අපි React-level optimization වලට useMemo, useCallback, සහ React.memo වගේ දේවල් ඉගෙන ගත්තා. හැබැයි Sprint 12 එකේදී අපි කතා කරන්නේ ඊට වඩා පුළුල් දෙයක් ගැන. ඒ කියන්නේ browser එකක් අපේ code එක pixels බවට හරවන්නේ කොහොමද, pages slow වෙන්නේ ඇයි, සහ user interaction එකක් delay වෙන්නේ ඇයි කියන එකයි.

1. Sprint 12 Goal

- **Concept Explanation (Sinhala):** Sprint 12 එකේ ප්‍රධානම බලාපොරොත්තුව තමයි web performance වල මූලික පදනම (performance foundations) තේරුම් ගැනීම. මේ sprint එක ඉවර වෙනකොට ඔයාට Real Performance සහ Perceived Performance අතර වෙනස පැහැදිලි කරන්න පුළුවන් වෙන්න ඕනේ. ඒ වගේම browser rendering process එක (DOM, CSSOM, Render Tree, Layout, Paint, Compositing) සිදුවන ආකාරයත්, Critical Rendering Path එක optimized කරගන්නේ කොහොමද කියලත් ඉගෙන ගන්න ලැබෙනවා. මීට අමතරව Layout Shift, CDN, Lazy Loading, Image Optimization, INP (Interaction to Next Paint) සහ performance වල බලපෑම පරිසරයට දැනෙන විදිහ (Sustainability) ගැනත් සරලව අවබෝධ කරගැනීම තමයි මෙහි ඉලක්කය. සරලවම කිව්වොත්, React app එකක් වැඩ කරපු පලියටම වැඩේ ඉවර වෙන්නේ නැහැ, ඒක fast වෙන්නත් ඕනේ.

2. Big Mental Model

- **Concept Explanation (Sinhala):** User කෙනෙක්ට අපේ React app එක පේන්න සහ පාවිච්චි කරන්න කලින් ඒක පියවර කිහිපයක් හරහා ගමන් කරනවා. browser එක මුලින්ම HTML, CSS, සහ JS download කරගන්නවා. ඊටපස්සේ browser එක ඇතුළෙන්ම structure එකක් හදාගෙන React run කරනවා. ඊටපස්සේ layout එක calculate කරලා, pixels paint කරලා තමයි screen එකට දාන්නේ. User button එකක් click කරපු ගමන් JavaScript සහ React ඒකට response එකක් හදනවා, ඊටපස්සේ browser එක ආයෙත් updated UI එක paint කරනවා. මේ මුළු process එකම කාර්යක්ෂමව (efficient) සිද්ධ වෙන එක තමයි හොඳ performance කියන්නේ.

3. What Is Web Performance?

- **Concept Explanation (Sinhala):** Web performance කියන්නේ website එකක් හෝ app එකක් කොච්චර ඉක්මනට load වෙනවද, display වෙනවද, සහ user click කරනකොට responsive වෙනවද කියන එකයි.

  හොඳ Performance තියෙන app එකක ලක්ෂණ:

  - වැදගත් content ඉක්මනින් පෙන්වීම.
  - Clicks සහ typing වලට ක්ෂණිකව response කිරීම.
  - Page එක load වෙනකොට elements එහා මෙහා පනින්නේ නැති වීම (no layout jumps).
  - Interaction එකක් වෙලාවේදී page එක freeze නොවීම.

  දුර්වල Performance තියෙන app එකක ලක්ෂණ:

  - Blank screen එකක් ගොඩක් වෙලා පෙන්වීම.
  - Content පෙන්වන්න කලින් විශාල JavaScript bundle එකක් load වෙනකන් බලා සිටීම.
  - Images load වෙනකොට content එක එහා මෙහා jump වීම.
  - Button එකක් click කරපු ගමන් මුළු page එකම freeze වීම.

4. Real Performance

- **Concept Explanation (Sinhala):** Real Performance කියන්නේ අපිට tools පාවිච්චි කරලා කෙලින්ම සංඛ්‍යාත්මකව (numbers වලින්) මැනගන්න පුළුවන් speed එකටයි. උදාහරණයක් විදිහට, "මේ page එක load වෙන්න තත්පර 1.8ක් ගියා", "JavaScript bundle එක 300KB වෙනවා", "server response time එක 200ms වුණා" වගේ දේවල් අපිට මනින්න පුළුවන්. Chrome DevTools, Lighthouse, Network tab, Performance tab සහ PageSpeed Insights වගේ tools පාවිච්චි කරලා අපිට මේවා මැනගන්න පුළුවන්.

5. Perceived Performance

- **Concept Explanation (Sinhala):** Perceived Performance කියන්නේ ඇත්තටම මනින වේගය නෙමෙයි, user කෙනෙක්ට අපේ app එක කොච්චර වේගවත්ද කියලා "දැනෙන" විදිහයි (user's feeling of speed). සමහර වෙලාවට page එකක් load වෙන්න තත්පර කිහිපයක් ගියත්, අපි මුලින්ම loading UI එකක් හෝ skeleton screen එකක් පෙන්නුවොත් user ට දැනෙන්නේ app එක ඉක්මනට වැඩ කරනවා වගේ. හැබැයි page එක ඉක්මනට load වුණත්, user button එකක් click කරාම කිසිම feedback එකක් නැතුව ටික වෙලාවක් blank වෙලා තිබුණොත්, user හිතන්නේ app එක slow නැත්නම් broken කියලා.
- **Code Block:**

```
// Perceived Performance එක වැඩි කරන්න හොඳ feedback pattern එකක්
<button disabled={isSaving}>
  {isSaving ? "Saving..." : "Save"}
</button>
```

6. Real Performance vs Perceived Performance

- **Concept Explanation (Sinhala):** Real Performance සහ Perceived Performance කියන දෙකම සාර්ථක app එකකට අත්‍යවශ්‍යයි. මේ දෙකේ වෙනස පහත වගුවෙන් ලෙහෙසියෙන්ම තේරුම් ගන්න පුළුවන්.

Topic	Meaning	Example
Real performance	Measurable speed (මැනිය හැකි වේගය)	Page loaded in 2 seconds
Perceived performance	How fast it feels (දැනෙන වේගය)	User sees a skeleton screen quickly

7. Browser Rendering: How Code Becomes Pixels

- **Concept Explanation (Sinhala):** browser එකකට HTML, CSS, සහ JS code ලැබුණාට පස්සේ ඒක එකපාරටම screen එකේ පෙන්නන්න බැහැ. ඒක පියවර කිහිපයක process එකක් හරහා යන්න ඕනේ. මේ මුළු process එකටම කියන්නේ Browser Rendering Flow එක කියලයි:
  1. HTML parsed වෙනවා -> DOM හැදෙනවා.
  2. CSS parsed වෙනවා -> CSSOM හැදෙනවා.
  3. DOM සහ CSSOM එකතු වෙලා -> Render Tree එක හැදෙනවා.
  4. Layout step එකෙන් elements වල size සහ position calculate කරනවා.
  5. Paint step එකෙන් pixels draw කරනවා.
  6. Composite step එකෙන් layers ඔක්කොම එකතු කරලා screen එකට දානවා.

8. Step 1: HTML Becomes the DOM

- **Concept Explanation (Sinhala):** HTML කියන්නේ page එකේ structure එක ලියන text code එකයි. browser එක මේ HTML කියවලා, ඒක තේරුම් ගන්න පුළුවන් structured tree-like representation එකකට හරවනවා. අන්න ඒකට තමයි DOM (Document Object Model) එක කියන්නේ. React වැඩ කරන්නේ මේ DOM එක update කරන්නයි.

9. Step 2: CSS Becomes the CSSOM

- **Concept Explanation (Sinhala):** CSS වලින් කියන්නේ elements පෙනෙන්න ඕන විදිහයි. HTML වලින් DOM එක හදනවා වගේම, browser එක CSS rules කියවලා CSSOM (CSS Object Model) එක හදාගන්නවා. browser එකට page එකක් render කරන්න නම් DOM එක වගේම CSSOM එකත් අත්‍යවශ්‍යයි. CSS files විශාල වුණොත් rendering delay වෙන්න පුළුවන්.

10. Step 3: DOM + CSSOM Become the Render Tree

- **Concept Explanation (Sinhala):** browser එක DOM (structure) සහ CSSOM (styles) දෙකම එකතු කරලා Render Tree එක හදාගන්නවා. Render Tree එකේ තියෙන්නේ screen එකේ ඇත්තටම පෙන්වන්න ඕන කරන visible elements විතරයි. උදාහරණයක් විදිහට, css වල `display: none` දාලා තියෙන element එකක් DOM එකේ තිබුණත් Render Tree එකට එකතු වෙන්නේ නැහැ, මොකද ඒක screen එකේ අඳින්න අවශ්‍ය නැති නිසා.

11. Step 4: Layout (Reflow)

- **Concept Explanation (Sinhala):** Layout (නැතහොත් Reflow) කියන පියවරේදී browser එක මගින් හැම element එකක්ම screen එකේ කොච්චර ඉඩක් ගන්නවද (width & height) සහ ඒක පිහිටන්න ඕන තැන (coordinates) calculate කරනවා. Images වල size කලින්ම දාලා නැති වුණොත්, නැත්නම් JS වලින් element size නිතරම වෙනස් කරොත් මේ layout calculation එක ආයේ ආයේ කරන්න වෙනවා. ඒක browser එකට ලොකු workload එකක් (expensive operation එකක්).

12. Step 5: Paint

- **Concept Explanation (Sinhala):** Layout එක හැදුනට පස්සේ browser එක ඇත්තටම screen එකේ pixels ටික පාට කරන්න පටන් ගන්නවා. මෙතැනදී text, colors, borders, shadows, backgrounds වගේ දේවල් අඳිනවා. ගොඩක් complex gradients, heavy shadows සහ විශාල images තියෙනවා නම් මේ paint කරන පියවරට වැඩි වෙලාවක් යනවා.

13. Step 6: Compositing

- **Concept Explanation (Sinhala):** browser එක සමහර elements (උදාහරණයක් විදිහට fixed headers, animations, overlays වැනි දේවල්) වෙන වෙනම layers විදිහට තමයි paint කරන්නේ. Compositing පියවරේදී සිද්ධ වෙන්නේ මේ වෙන වෙනම paint කරපු layers ඔක්කොම නිවැරදි පිළිවෙළට එකතු කරලා final screen එක user ට පෙන්වන එකයි.

14. Browser Rendering Summary

- **Concept Explanation (Sinhala):** කෙටියෙන් මතක තියාගන්න rendering flow එක පහත පරිදි සරලව සාරාංශ කරන්න පුළුවන්:

```
DOM (Structure) → CSSOM (Styles) → Render Tree (Visible styled
elements) → Layout (Size & Position) → Paint (Pixels drawing) →
Composite (Final screen)
```

15. Critical Rendering Path

- **Concept Explanation (Sinhala):** Critical Rendering Path කියන්නේ user ට screen එකේ පළමු වැදගත් pixels ටික (first useful content) පේන්න කලින් browser එකට අනිවාර්යයෙන්ම download කරලා run කරන්න වෙන අවම වැඩ ප්‍රමාණයයි (minimum path). අපි නොවැදගත් heavy scripts මුලින්ම load වෙන්න හැදුවොත්, browser එක content එක පෙන්වන්නේ නැතුව ඒ script එක download වෙනකන් බලාගෙන ඉන්නවා. ඒක නිසා වැදගත් code මුලින්ම load කරලා, අනෙක්වා පස්සේ load වෙන්න සැලසුම් කරන්න ඕනේ.
- **Code Block:**

```
<!-- BAD: නොවැදගත් heavy script එකක් නිසා render එක block වෙනවා -->
<script src="large-analytics-file.js"></script>
<h1>Welcome</h1>
<p>Main content</p>

<!-- BETTER: script එක defer කරලා තියෙන නිසා content එක ඉක්මනට පේනවා -->
<h1>Welcome</h1>
<p>Main content appears early.</p>
<script src="large-analytics-file.js" defer></script>
```

16. Source Order

- **Concept Explanation (Sinhala):** Source order කියන්නේ අපේ HTML හෝ JSX code එකේ elements ලියලා තියෙන පිළිවෙළයි. browser එක code එක කියවන්නේ උඩ ඉඳන් පල්ලෙහාට නිසා, වැදගත්ම content එක හැමවෙලේම code එකේ ඉහළින්ම තියෙන්න ඕනේ. Heavy charts වගේ දේවල් ඊට පල්ලෙහායින් තියන්න පුළුවන්.
- **Code Block:**

```
// BETTER: වැදගත් Dashboard summary එක මුලින්ම තියෙන නිසා ඉක්මනින් පේනවා
<main>
  <h1>Dashboard</h1>
  <p>Important summary appears first.</p>
  <LargeChart />
</main>
```

17. Progressive Enhancement

- **Concept Explanation (Sinhala):** Progressive enhancement කියන්නේ සරලවම මුලින්ම basic වැඩ කරන UI එකක් user ට පෙන්වලා, ඊටපස්සේ පසුබිමෙන් advanced JavaScript load වෙනකොට extra interactive features එකතු කරන ක්‍රමයටයි. මේකෙන් slow network තියෙන අයට වගේම අඩු පහසුකම් තියෙන mobile phones පාවිච්චි කරන අයටත් කිසිම කරදරයක් නැතුව page එකේ මූලික තොරතුරු ටික ඉක්මනින් බලාගන්න පුළුවන් වෙනවා.

18. Render-Blocking Resources

- **Concept Explanation (Sinhala):** browser එකට page එක render කරන්න නොදී මගදී නවත්තලා තියෙන resources වලට Render-Blocking Resources කියලා කියනවා. විශාල CSS files සහ async නොවන සාමාන්‍ය JavaScript tags මේකට ප්‍රධානම උදාහරණයි. browser එකට `<script>` tag එකක් හමුවුණාම HTML කියවන එක නතර කරලා ඒ script එක run කරන්න පටන් ගන්නවා. ඒක නිසා page එක render වීම ප්‍රමාද වෙනවා.

19. The defer Attribute

- **Concept Explanation (Sinhala):** `<script>` tag එකකට `defer` attribute එක දැම්මහම සිද්ධ වෙන්නේ browser එක HTML parse කරන එක නතර කරන්නේ නැතුව, background එකෙන් script එක download කරනවා. ඊටපස්සේ HTML parse කරලා ඉවර වුණාට පස්සේ තමයි ඒ script එක run කරන්නේ. මුලින්ම render වෙන්න අවශ්‍ය නැති analytics scripts, widgets වගේ දේවල් වලට මේක අනිවාර්යයෙන්ම පාවිච්චි කරන්න ඕනේ.
- **Code Block:**

```
<script src="analytics.js" defer></script>
```

20. Layout Shift

- **Concept Explanation (Sinhala):** Layout Shift කියන්නේ page එක load වෙමින් පවතින අතරතුරේදී එහි ඇති content එක එකපාරටම පහළට හෝ වෙනත් පැත්තකට jump වීමයි (unexpected movement). මේක ගොඩක් වෙලාවට වෙන්නේ images වල width සහ height කලින්ම සීමා කරලා නැති වුණාමයි. image එක load වෙලා ඉවර වුණ ගමන් ඒක විශාල ඉඩක් ගන්න නිසා ඊට යටින් තියෙන text එකපාරටම පහළට පනිනවා. මේක user experience එකට ඉතාමත් අහිතකරයි.
- **Code Block:**

```
// BAD: image size එක කලින්ම දාන්නේ නැති නිසා layout jump එකක් වෙනවා
<img src="hero.jpg" alt="Dashboard preview" />

// BETTER: width & height දාලා තියෙන නිසා image එක load වෙන්න කලින්ම ඉඩ
reserve වෙනවා
<img
  src="hero.webp"
  width="800"
  height="450"
  alt="Dashboard preview"
  loading="lazy"
/>
```

21. Image Optimization

- **Concept Explanation (Sinhala):** Images කියන්නේ සාමාන්‍යයෙන් website එකක තියෙන ලොකුම files ජාතියක්. ඒක නිසා images optimize කරන එක performance වලට ගොඩක් වැදගත්. හැමවෙලේම PNG වෙනුවට WebP වගේ modern light-weight image formats පාවිච්චි කරන්න ඕනේ. ඒ වගේම image එක screen එකේ පෙන්වන size එකට ගැලපෙන resolution එකක් තියෙන image එකක්ම පාවිච්චි කරන්න පුරුදු වෙන්න ඕනේ (අනවශ්‍ය ලෙස 5000px වගේ විශාල images upload කරන්න එපා).

22. Lazy Loading

- **Concept Explanation (Sinhala):** Lazy Loading කියන්නේ යම්කිසි content එකක් (විශේෂයෙන්ම images හෝ බර components) ඇත්තටම අවශ්‍ය වෙනකන් (උදාහරණයක් විදිහට user scroll කරලා ඒ element එක තියෙන තැනට එනකන්) load නොකර පමා කරලා තැබීමයි. Image එකකට `loading="lazy"` දැම්මම screen එකෙන් පිටත තියෙන images මුලදීම download වෙන්නේ නැහැ. React වලදී අපිට React `lazy` සහ `Suspense` පාවිච්චි කරලා මුළු routes/components වුණත් lazy load කරන්න පුළුවන්.
- **Code Block:**

```
// React Component Lazy Loading උදාහරණයක්
import { lazy, Suspense } from "react";
const Dashboard = lazy(() => import("./pages/Dashboard"));

function App() {
  return (
    <Suspense fallback={<p>Loading dashboard...</p>}>
      <Dashboard />
    </Suspense>
  );
}
```

23. Latency

- **Concept Explanation (Sinhala):** Latency කියන්නේ සරලවම "travel time" එකයි. ඒ කියන්නේ user ගේ browser එකෙන් server එකට request එකක් ගිහින්, ඒක process වෙලා ආපසු response එකක් browser එකට ලැබෙන්න යන කාලයයි. අපේ frontend React code එක කොච්චර සුපිරියට ලිව්වත්, server එක ලංකාවෙන් ගොඩක් ඈත (උදාහරණයක් විදිහට US වල) තියෙනවා නම් latency එක වැඩි නිසා app එක slow කියලා user ට දැනෙනවා.

24. CDN (Content Delivery Network)

- **Concept Explanation (Sinhala):** CDN එකක් කියන්නේ ලෝකය පුරා විසිරිලා තියෙන servers ජාලයකටයි. අපේ website එකේ static assets (images, css, js files, fonts) මේ CDN එකක දාලා තිබ්බොත්, ලංකාවේ ඉන්න user කෙනෙක් අපේ web page එකට එනකොට US වල තියෙන server එකට යන්නේ නැතුව, ලංකාවට ළඟම තියෙන CDN server එකකින් (edge location) ඒ files ඉක්මනට download කරලා දෙනවා. මේකෙන් latency එක ගොඩක් අඩු වෙනවා.

25. Compression

- **Concept Explanation (Sinhala):** Compression කියන්නේ files වල size එක කුඩා කරලා network එක හරහා යවන ක්‍රමයටයි (Gzip හෝ Brotli වගේ algorithms පාවිච්චි කරලා). Files කුඩා වෙනකොට download වෙන්න යන වෙලාව අඩු වෙනවා, bandwidth එක ඉතිරි වෙනවා, වගේම mobile users ලට ඉක්මනින්ම page එක load වෙනවා.

26. Caching

- **Concept Explanation (Sinhala):** Caching කියන්නේ යම්කිසි resource එකක් (css, js, image) browser එක ඇතුළේ copy එකක් විදිහට save කරලා තියාගන්න එකයි. User දෙවැනි පාරටත් අපේ site එකට එනකොට ඒ files ආයේ server එකෙන් download කරන්නේ නැතුව තමන්ගේ browser cache එකෙන්ම අරගෙන පාවිච්චි කරනවා. මේකෙන් repeat visits සුපිරි වේගයෙන් සිද්ධ වෙනවා.

27. Time to Usable

- **Concept Explanation (Sinhala):** Time to Usable කියන්නේ page එක screen එකේ පෙනෙන්න පටන් ගන්නවා විතරක් නෙමෙයි, user ට ඒක ඇත්තටම click කරලා වැඩක් කරන්න පුළුවන් තත්ත්වයට පත්වෙන්න යන කාලයයි. සමහර වෙලාවට page එකේ content පෙනුනත් JavaScript එක background එකේ බර වැඩක් කර කර ඉන්නවා නම්, user button එකක් click කරාට ඒක වැඩ කරන්නේ නැහැ. ඒක නිසා heavy JS වැඩ පස්සට කල් දාලා Time to Usable කාලය අඩු කරගන්න ඕනේ.

28. INP: Interaction to Next Paint

- **Concept Explanation (Sinhala):** INP (Interaction to Next Paint) කියන්නේ user කෙනෙක් මොනවා හරි interaction එකක් (click එකක්, key press එකක්) කරාට පස්සේ, browser එකට ඊළඟ visual update එක (next paint) screen එකට දාන්න යන මුළු කාලයයි.
  - **Good INP:** 200ms වලට වඩා අඩු කාලයක්. (User ට ක්ෂණිකව UI එක update වෙනවා දැනෙනවා).
  - **Poor INP:** 500ms වලට වඩා වැඩි කාලයක්. (UI එක හිරවෙලා වගේ දැනෙනවා).

29. Why Heavy JavaScript Hurts Interaction

- **Concept Explanation (Sinhala):** browser එකේ style calculation, layout, paint සහ අපේ JavaScript event handlers ඔක්කොම දුවන්නේ එකම "Main Thread" එකක් ඇතුළෙයි. අපි button click එකක් ඇතුළේ ලොකු `for` loop එකක් වගේ බර JavaScript වැඩක් දුවන්න හැදුවොත්, browser එක ඒ වැඩේ ඉවර වෙනකන් වෙන කිසිම UI update එකක් කරන්නේ නැහැ. ඒ වෙලාවට user ට app එක freeze වෙලා වගේ දැනෙනවා.
- **Code Block:**

```
// BAD: Click එකක් වෙලාවෙ Main Thread එක block කරන බර වැඩක්
export default function App() {
  function handleSearch() {
    let total = 0;
    for (let i = 0; i < 2_000_000_000; i++) {
      total += i;
    }
    console.log(total);
  }

  return (
    <button type="button" onClick={handleSearch}>
      Search
    </button>
  );
}
```

30. Fast Feedback During Interaction

- **Concept Explanation (Sinhala):** යම්කිසි ක්‍රියාවකට (action) සැබෑවටම ප්‍රතිඵලයක් ලැබෙන්න වෙලාවක් යනවා නම් (උදාහරණයකට API call එකක් වගේ), අපි user ට ඒක පටන් ගත්ත ගමන් ක්ෂණිකව feedback එකක් දෙන්න ඕනේ. "Searching..." හෝ "Saving..." වැනි status text එකක් පෙන්වීමෙන් user දන්නවා app එක වැඩ කරනවා කියලා.
- **Code Block:**

```
import { useState } from "react";

export default function App() {
  const [status, setStatus] = useState("idle");

  function handleSearch() {
    setStatus("Searching...");
    setTimeout(() => {
      setStatus("Search complete");
    }, 500);
  }

  return (
    <main>
      <button type="button" onClick={handleSearch}>
        Search
      </button>
      <p>{status}</p>
    </main>
  );
}
```

31. Skeleton Screens and Loading UI

- **Concept Explanation (Sinhala):** Loading UI එකක් කියන්නේ "Loading..." වගේ සරල text එකක් පෙන්වන එකයි. Skeleton Screen එකක් කියන්නේ ඊටත් වඩා දියුණු ක්‍රමයක්. මෙතැනදී ඇත්ත content එක එන්න කලින් ඒ content එකේ හැඩය ගත්තු (gray color placeholder boxes) screen එකක් පෙන්වනවා. මේකෙන් user ට දැනෙන්නේ app එක ගොඩක් responsive සහ දැනටමත් data load වෙලා ඉවරයි වගේ හැඟීමක්.

32. Avoiding Layout Jumps

- **Concept Explanation (Sinhala):** Layout jumps වලක්වා ගන්න නම් අපි හැමවෙලේම කලින්ම elements වලට අදාළ ඉඩ වෙන් කරලා තියන්න ඕනේ. උදාහරණයකට images වලට exact `width` සහ `height` දීම, late load වෙන ads හෝ embeds වලට placeholder boxes හදා තිබීම වැනි දේවල් කරන්න පුළුවන්.

33. Performance and Accessibility

- **Concept Explanation (Sinhala):** Performance සහ Accessibility (පහසුවෙන් ප්‍රවේශ විය හැකි බව) කියන්නේ එකිනෙකට තදින්ම බැඳුණු සංකල්ප දෙකක්. Page එක slow වෙනකොට හෝ layout jumps වෙනකොට keyboard navigation පාවිච්චි කරන අයට සහ screen readers පාවිච්චි කරන පෙනීමේ අපහසුතා ඇති අයට තමන් හිටපු තැන නැතිවෙලා ලොකු අපහසුතාවයකට මුහුණ දෙන්න වෙනවා. ඒ නිසා responsive සහ stable UI එකක් හදන එක සැමටම සාධාරණයි.

34. Sustainability and Performance

- **Concept Explanation (Sinhala):** අපි හදන බර වැඩි (heavy) code නිසා user ගේ device එකේ CPU එකට වැඩිපුර වැඩ කරන්න වෙනවා. ඒ නිසා battery එක ඉක්මනට බහිනවා, server resources වැඩිපුර වැය වෙනවා, වගේම අවසානයේදී ලෝකයේ විදුලි පරිභෝජනය වැඩි වෙලා කාබන් විමෝචනය (carbon emissions) වැඩි වෙනවා. සරලවම, අපේ app එකක් optimize කරලා size එක අඩු කරන එකෙන් අපි පරිසරය සුරකින්නත් දායක වෙනවා.

35. Common Beginner Performance Problems

- **Concept Explanation (Sinhala):** මෙන්න නවක developers ලා නිතරම කරන වැරදි සහ ඒවා නිවැරදි කරගන්නා ආකාරය පිළිබඳ සරල සංසන්දනයක්.
- **Code Block:**

```
// 1. Huge Images වෙනුවට Optimized Images පාවිච්චිය
// BAD:
<img src="large-5000px-photo.png" alt="Product" />
// BETTER:
<img src="product.webp" width="400" height="300" alt="Product"
loading="lazy" />

// 2. Loading State නොමැති වීම වෙනුවට Fast Feedback දීම
// BAD:
<button type="button" onClick={handleSave}>Save</button>
// BETTER:
<button type="button" disabled={isSaving} onClick={handleSave}>
  {isSaving ? "Saving..." : "Save"}
</button>

// 3. Render-blocking script වෙනුවට defer පාවිච්චිය
// BAD:
<script src="large-non-critical.js"></script>
<h1>Welcome</h1>
// BETTER:
<h1>Welcome</h1>
<script src="large-non-critical.js" defer></script>
```

36. Beginner Chrome DevTools Performance Checks

- **Concept Explanation (Sinhala):** අපිට performance මනින්න පුළුවන් ප්‍රධාන ක්‍රම 3ක් තියෙනවා:
  1. **Network Tab:** මෙතැනින් අපිට බලාගන්න පුළුවන් load වෙන files මොනවද, ඒවායේ sizes කොහොමද, සහ slow වෙන files මොනවද කියලා.
  2. **Performance Tab:** මෙතැනින් user click කරනකොට main thread එක block වෙනවද, JavaScript එකට කොච්චර වෙලාවක් යනවද කියලා record කරලා බලාගන්න පුළුවන්.
  3. **Lighthouse:** මේකෙන් අපේ මුළු site එකටම automatic report එකක් සහ score එකක් ලබාගන්න පුළුවන්.

37. Sprint 12 Mini Project & Code (App.jsx & App.css)

- **Concept Explanation (Sinhala):** මේ mini project එකෙන් අපි කලින් ඉගෙනගත්ත theory ටික ප්‍රායෝගිකව අත්හදා බලනවා. මෙතැනදී වැදගත් content මුලින්ම පෙන්වනවා, instant feedback button එකක් තියෙනවා, lazy dynamic image එකක් තියෙනවා, සහ heavy-work demo button එකක් තියෙනවා.
- **Code Block (src/App.jsx):**

```
import { useState } from "react";
import "./App.css";

export default function App() {
  const [feedback, setFeedback] = useState("No interaction yet.");
  const [heavyResult, setHeavyResult] = useState(null);

  function handleFastFeedback() {
    setFeedback("Button responded immediately.");
  }

  function handleHeavyWork() {
    setFeedback("Running heavy work...");

    // browser එකේ main thread එක හිතාමතාම block කරන loop එකක්
    let total = 0;
    for (let i = 0; i < 50_000_000; i++) {
      total += i;
    }
    setHeavyResult(total);
    setFeedback("Heavy work finished.");
  }

  return (
    <main className="page">
      <section className="hero">
        <h1>Performance Notes</h1>
        <p>
          A React app should not only work correctly. It should also
load,
          render, and respond quickly.
        </p>
      </section>

      <section className="card">
        <h2>Real Performance</h2>
        <p>
          Real performance is measured with numbers, such as load time,
          response time, and interaction delay.
        </p>
      </section>

      <section className="card">
        <h2>Perceived Performance</h2>
        <p>
          Perceived performance is how fast the app feels to the user.
Loading
          messages and instant feedback improve the feeling.
        </p>
        <button type="button" onClick={handleFastFeedback}>
          Give Fast Feedback
        </button>
        <p className="status">{feedback}</p>
      </section>

      <section className="card">
        <h2>Lazy Image</h2>
        <p>
          This image reserves space with width and height, and it uses
lazy
          loading.
        </p>
        <img
          src="[https://placehold.co/800x450]
(https://placehold.co/800x450)"
          width="800"
          height="450"
          alt="Placeholder dashboard preview"
          loading="lazy"
          className="preview"
        />
      </section>

      <section className="card">
        <h2>Interaction Delay Demo</h2>
        <p>
          Click this button and observe whether the page feels delayed.
Then
          record it in Chrome DevTools Performance tab.
        </p>
        <button type="button" onClick={handleHeavyWork}>
          Run Heavy Work
        </button>
        {heavyResult !== null && <p>Result: {heavyResult}</p>}
      </section>
    </main>
  );
}
```

- **Code Block (src/App.css):**

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f5f5f5;
  color: #222;
}

.page {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px;
}

.hero {
  margin-bottom: 24px;
}

.card {
  background: white;
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 16px;
  margin-bottom: 16px;
}

button {
  border: none;
  border-radius: 8px;
  padding: 10px 14px;
  cursor: pointer;
  background-color: #0070f3;
  color: white;
  font-weight: 600;
}

button:hover {
  background-color: #0051a8;
}

.status {
  font-weight: 600;
  color: #333;
}

.preview {
  display: block;
  max-width: 100%;
  height: auto;
  border-radius: 8px;
}
```

41. Debug Task

- **Concept Explanation (Sinhala):** පහත වැරදි සහිත code එක නිවැරදි කරලා හොඳ performance තියෙන React-friendly code එකක් බවට පත් කරන හැටි බලමු.
- **Code Block:**

```
<!-- BAD HTML VERSION -->
<img src="photo.png" alt="Photo" />
<script src="large-non-critical.js"></script>
<button onclick="doVeryHeavyWork()">Search</button>

<!-- FIXED HTML VERSION -->
<img
  src="photo.webp"
  width="800"
  height="450"
  alt="Dashboard preview"
  loading="lazy"
/>
<script src="large-non-critical.js" defer></script>
<button type="button" id="search-button">
  Search
</button>
```

42. Common Sprint 12 Vocabulary

- **Concept Explanation (Sinhala):** මෙන්න මේ sprint එකේදී අපිට නිතරම හමුවන වැදගත්ම වචන ටික සරල සිංහල තේරුම් සමඟ:

Term	Simple Meaning (සිංහල තේරුම)
Real performance	Speed measured with numbers (නියම මැනිය හැකි වේගය)
Perceived performance	How fast the app feels (user ට දැනෙන වේගය)
DOM	Browser structure created from HTML (HTML වලින් සාදන ව්‍යුහය)
CSSOM	Browser structure created from CSS (CSS වලින් සාදන ව්‍යුහය)
Render tree	Visible styled elements the browser plans to draw (ඇඳීමට නියමිත elements)
Layout (Reflow)	Browser calculates size and position (ප්‍රමාණය සහ පිහිටීම මැනීම)
Paint	Browser draws pixels (pixels පාට කිරීම)
Composite	Browser combines layers into final screen (layers එකතු කිරීම)
Critical rendering path	Work needed before useful content appears (content පෙන්වීමට පෙර අත්‍යවශ්‍ය වැඩ කොටස)
Layout shift	Unexpected movement of content (content එකපාරටම එහා මෙහා පැනීම)
Lazy loading	Loading something only when needed (අවශ්‍ය වූ විට පමණක් load කිරීම)
Latency	Network travel delay (network එක හරහා දත්ත යාමට එන්න ගතවන ප්‍රමාදය)
INP	Delay from interaction to next visual update (interaction එකක සිට ඊළඟ visual update එකට යන ප්‍රමාදය)

43. Revision Checkpoint Questions

- **Concept Explanation (Sinhala):** ඔයාට තනියම පාඩම් කරලා මතකය පරීක්ෂා කරගන්න පහත ප්‍රශ්න ටික උදව් වේවි. (මේවාට උත්තර හිතින් හරි සිංහලෙන් හරි හිතන්න උත්සාහ කරන්න):
  1. Real performance කියන්නේ මොකක්ද?
  2. Perceived performance කියන්නේ මොකක්ද?
  3. browser එකක් HTML, CSS කියවලා අන්තිමට screen එකට දානකන් වෙන process එකේ පියවර මොනවාද?
  4. DOM සහ CSSOM අතර වෙනස මොකක්ද?
  5. Layout (Reflow) කියන පියවර slow වෙන්න බලපාන හේතු මොනවාද?
  6. Critical Rendering Path එක optimized කරන්න `<script>` tag එකකට දාන්න පුළුවන් attribute එක මොකක්ද?
  7. Layout Shift කියන්නේ මොකක්ද? ඒක වලක්වන්නේ කොහොමද?
  8. Lazy Loading පාවිච්චි කරන්නේ ඇයි?
  9. INP (Interaction to Next Paint) මනින්නේ මොකක්ද?
  10. Website එකක performance වැඩි කරනකොට ඒකෙන් පරිසරයට (Sustainability) උදව් වෙන්නේ කොහොමද?